SELENIUM FINGERTIPS
THE COMPLETE QUICK REFERENCE

Every locator, command, wait, class and combo - one page-flip away

Prepared for: jashu5446
29 September 2026

INDEX

SELENIUM FINGERTIPS — THE COMPLETE QUICK REFERENCE 3

SECTION 1 — LOCATORS (the 8 ways to FIND) 4

SECTION 2 — DRIVER COMMANDS (the browser level) 5

SECTION 3 — WEBELEMENT ACTIONS (the element level) 6

SECTION 4 — TEXT & ATTRIBUTE READERS (know the differences!) 7

SECTION 5 — WAITS (the 3 + the conditions) 8

SECTION 6 — WINDOWS, FRAMES & ALERTS (switchTo — the 3 worlds) 11

SECTION 7 — ACTIONS CLASS (mouse + keyboard gestures) 12

SECTION 8 — SELECT, JAVASCRIPT & SCREENSHOT (the 3 helper interfaces) 13

SECTION 9 — THE BUILT-IN CLASSES & INTERFACES (know each by NAME) 14

SECTION 10 — THE TESTNG TOOLKIT (annotations, attributes, asserts) 15

SECTION 11 — LISTENERS & SCREENSHOT ON FAILURE (the code they ask for) 16

SECTION 12 — COMMON EXCEPTIONS (the quick table) 18

SECTION 13 — COMMANDS (Maven, Git, Grid — the daily ones) 19

SECTION 14 — POWER COMBOS (the daily recipes — memorize these five) 20

SELENIUM FINGERTIPS — THE COMPLETE QUICK REFERENCE

Every locator, action, class, wait, assertion and command — one line each, with an example and a pro-tip to memorize it.

SECTION 1 — LOCATORS (the 8 ways to FIND)

Locator	Finds by	Example
`By.id()`	the id attribute — FASTEST, first choice	`driver.findElement(By.id("username"))`
`By.name()`	the name attribute	`driver.findElement(By.name("email"))`
`By.className()`	a CSS class	`driver.findElement(By.className("btn-primary"))`
`By.tagName()`	the tag itself	`driver.findElements(By.tagName("a"))`
`By.linkText()`	EXACT link text	`driver.findElement(By.linkText("Forgot Password?"))`
`By.partialLinkText()`	PART of link text	`driver.findElement(By.partialLinkText("Forgot"))`
`By.cssSelector()`	CSS (fast, goes DOWN only)	`driver.findElement(By.cssSelector("input#email"))`
`By.xpath()`	XPath (powerful, goes UP too)	`driver.findElement(By.xpath("//button[@id='submit']"))`

XPath helpers for dynamic elements:

Trick	Syntax	Use
contains	`//input[contains(@id,'user')]`	dynamic ids like user_83721
starts-with	`//input[starts-with(@id,'user_')]`	stable prefix
text	`//button[text()='Login']`	find by label
parent axis	`//input[@id='x']//parent::div`	go UP (CSS cannot)
following-sibling	`//td[text()='Laptop']//following-sibling::td[1]`	table rows

MEMORY HOOK: *"ID is the King, XPath is the last resort — CSS goes down only, XPath goes up too."*

SECTION 2 — DRIVER COMMANDS (the browser level)

Command	Does	Example
`get()`	open a URL (waits for load)	`driver.get("https://app.com");`
`getCurrentUrl()`	read the URL	`if (driver.getCurrentUrl().contains("dashboard"))`
`getTitle()`	read the tab title	`Assert.assertEquals(driver.getTitle(), "Home");`
`getPageSource()`	full HTML of the page	`driver.getPageSource().contains("Welcome")`
`navigate().to()`	open (doesn't wait like get)	`driver.navigate().to("https://app.com");`
`navigate().back()`	browser back button	`driver.navigate().back();`
`navigate().forward()`	browser forward	`driver.navigate().forward();`
`navigate().refresh()`	F5	`driver.navigate().refresh();`
`manage().window().maximize()`	maximize	`driver.manage().window().maximize();`
`manage().window().setSize()`	exact size	`driver.manage().window().setSize(new Dimension(390, 844));`
`manage().timeouts()`	the 3 timeouts	see Section 5
`manage().deleteAllCookies()`	wipe cookies	`driver.manage().deleteAllCookies();`
`manage().addCookie()`	inject a cookie	`driver.manage().addCookie(new Cookie("session", "abc"));`
`manage().getCookies()`	read all cookies	`Set<Cookie> all = driver.manage().getCookies();`
`close()`	close CURRENT window only	`driver.close();`
`quit()`	close ALL + end session	`driver.quit();`

MEMORY HOOK: *"get waits, navigate remembers; close one, quit ALL + kills the session."*

SECTION 3 — WEBELEMENT ACTIONS (the element level)

Method	Does	Example
`click()`	click	`driver.findElement(By.id("login")).click();`
`sendKeys()`	type into an input	`driver.findElement(By.id("email")).sendKeys("a@b.com");`
`sendKeys(Keys...)`	special keys	`element.sendKeys(Keys.CONTROL + "a");`
`clear()`	empty a field	`element.clear();`
`submit()`	submit the FORM	`element.submit();`
`isDisplayed()`	visible? boolean	`if (banner.isDisplayed())`
`isEnabled()`	enabled? boolean	`Assert.assertTrue(submitBtn.isEnabled());`
`isSelected()`	radio/checkbox ticked?	`Assert.assertTrue(terms.isSelected());`
`getLocation()`	x,y position	`element.getLocation().getX()`
`getSize()`	dimensions	`element.getSize().getHeight()`
`getRect()`	position + size (Selenium 4)	`element.getRect()`
`findElement()`	child search	`row.findElement(By.tagName("td"))`
`findElements()`	ALL children matching	`List<WebElement> cells = row.findElements(...)`

MEMORY HOOK: *"click, type, clear, submit — then LOOK with isDisplayed/Enabled/Selected (booleans for if-checks, never for final assertions when a wait exists)."*

SECTION 4 — TEXT & ATTRIBUTE READERS (know the differences!)

Method	Returns	Reads from	Example
`getText()`	the VISIBLE text between tags	the page	`String msg = alert.getText();`
`getAttribute("value")`	what's TYPED in an input, or any attribute	inside the tag	`String typed = box.getAttribute("value");`
`getAttribute("href")`	any attribute's value	the tag	`link.getAttribute("href")`
`getDomAttribute()`	the plain HTML attribute (Selenium 4)	the tag	`element.getDomAttribute("data-role")`
`getDomProperty()`	the live JS property (Selenium 4)	the DOM object	`element.getDomProperty("value")`
`getCssValue()`	a computed style	CSS	`btn.getCssValue("color")`
`getTagName()`	the tag name	the tag	`el.getTagName()`

PRO TIP (the memorization rule):

getText = between the tags (what eyes see) — getAttribute = inside the tag (what the HTML says) — getAttribute("value") = what's typed — getCssValue = how it looks. *The classic trap:* `box.getText()` *returns EMPTY for an input field — inputs keep their content in the* `value` *ATTRIBUTE, not in text. Say that sentence in the interview and watch them nod.*

MEMORY HOOK: *"Text between, attribute inside, typed value in the value attribute."*

SECTION 5 — WAITS (the 3 + the conditions)

Wait	Scope	Example
implicit	global — all findElement calls	`driver.manage().timeouts().implicitlyWait(10, TimeUnit.SECONDS);`
explicit	One specific condition	`new WebDriverWait(driver, Duration.ofSeconds(10)).until(ExpectedConditions.elementToBeClickable(By.id("x")));`

Wait	Scope	Example
fluent	explicit-custom-polling-ignore	`new FluentWait<>(driver).withTimeout(30, SECONDS).pollingEvery(3, SECONDS).ignoring(NoSuchElementException.class);`
pageLoadTimeout	pageload-budget	`driver.manage().timeouts().pageLoadTimeout(30, SECONDS);`
setScriptTimeout	async-JS-budget	`driver.manage().timeouts().setScriptTimeout(10, SECONDS);`

Wait	Scope	Example
`Thread.sleep()`	BLIND sleep — debugging only	`Thread.sleep(3000);` (banned in suites)

The ExpectedConditions you actually use:

Condition	Waits until
`visibilityOfElementLocated(By)`	element is rendered
`elementToBeClickable(By)`	visible + enabled
`presenceOfElementLocated(By)`	in the DOM (may be hidden)
`invisibilityOfElementLocated(By)`	gone from view
`textToBePresentInElement(el, text)`	the text appears
`alertIsPresent()`	a JS alert shows
`frameToBeAvailableAndSwitchToIt(id)`	the frame is ready
`titleContains("Dash")`	the tab title

MEMORY HOOK: *"Implicit = blanket, explicit = sniper, fluent = sniper with a clock — presence in DOM, visibility on screen, clickability enabled too. Never mix blanket and sniper."*

SECTION 6 — WINDOWS, FRAMES & ALERTS (switchTo — the 3 worlds)

Method	Does	Example
`getWindowHandle()`	THIS window's id (String)	`String parent = driver.getWindowHandle();`
`getWindowHandles()`	ALL window ids (Set)	`for (String h : driver.getWindowHandles())`
`switchTo().window(handle)`	focus a window	`driver.switchTo().window(child);`
`switchTo().frame(index/name/element)`	enter an iframe	`driver.switchTo().frame(0);`
`switchTo().defaultContent()`	leave the frame	`driver.switchTo().defaultContent();`
`switchTo().parentFrame()`	one frame up	`driver.switchTo().parentFrame();`
`switchTo().alert()`	focus a JS popup	`Alert a = driver.switchTo().alert();`
`alert.accept()`	click OK	`a.accept();`
`alert.dismiss()`	click Cancel	`a.dismiss();`
`alert.getText()`	read the message	`String m = a.getText();`
`alert.sendKeys("x")`	type into a prompt	`a.sendKeys("hello");`

MEMORY HOOK: *"Handle = THIS (String), HandleS = ALL (Set) — the 'S' is everything. Frames are rooms: switchTo to enter, defaultContent to come home. Alerts are browser-owned: locators never work on them."*

SECTION 7 — ACTIONS CLASS (mouse + keyboard gestures)

Method	Does	Example
`moveToElement()`	HOVER (reveals menus)	`actions.moveToElement(menu).perform();`
`contextClick()`	right-click	`actions.contextClick(el).perform();`
`doubleClick()`	double-click	`actions.doubleClick(el).perform();`
`dragAndDrop(src, dest)`	drag and drop	`actions.dragAndDrop(s, d).perform();`
`clickAndHold()`	press without release	for custom drags
`moveByOffset(x, y)`	move to coordinates	hover sequences
`keyDown(Keys.SHIFT)`	hold a modifier	`actions.keyDown(Keys.SHIFT).click(a).keyUp(Keys.SHIFT).perform();`
`sendKeys()`	type during the chain	`actions.sendKeys("abc").perform();`

MEMORY HOOK: *"Mouse = Actions; every chain ends with .perform() or nothing happens."*

SECTION 8 — SELECT, JAVASCRIPT & SCREENSHOT (the 3 helper interfaces)

Select class (only for `<select>` tags):

Method	Does
`selectByVisibleText("India")`	choose by shown text
`selectByValue("in")`	choose by the value attribute
`selectByIndex(2)`	choose by position (0-based)
`getOptions()`	all options (count/verify them)
`getFirstSelectedOption()`	the current choice
`isMultiple()`	is it a multi-select

Fake dropdowns (div/ul/li): click to open + findElements + loop and click match.

JavascriptExecutor (when native Selenium fails):

Script	Use
`window.scrollBy(0, 500)`	scroll down
`window.scrollTo(0, document.body.scrollHeight)`	scroll to page end
`arguments[0].click()`	force a click past an overlay
`arguments[0].value='abc'`	set a value directly

TakesScreenshot (3 steps):

```
File src = ((TakesScreenshot) driver).getScreenshotAs(OutputType.FILE);
FileUtils.copyFile(src, new File("screens/" + testName + ".png"));
// Selenium 4 bonus: element-only screenshot:
File el = element.getScreenshotAs(OutputType.FILE);
```

MEMORY HOOK: *"Select for select tags, JS as last resort, screenshot = cast + capture + copy."*

SECTION 9 — THE BUILT-IN CLASSES & INTERFACES (know each by NAME)

Class / Interface	One-line identity
WebDriver	the interface every driver implements (`WebDriver driver = new ChromeDriver()`)
SearchContext	the parent interface — findElement/findElements only
WebElement	one located element — click, sendKeys, getText
By	the locator strategy factory (By.id, By.cssSelector...)
ChromeDriver / FirefoxDriver / EdgeDriver	the real browser drivers
RemoteWebDriver	drive a browser on a GRID via the hub URL
ChromeOptions / FirefoxOptions	headless, SSL bypass, prefs, binary
Timeouts (manage().timeouts())	the 3 timeouts in one interface
WebDriverWait	the explicit wait
FluentWait	wait with custom polling + ignore
ExpectedConditions	the ready-made wait conditions
Alert	the JS popup
Select	the dropdown helper
Actions	mouse + keyboard gestures
JavascriptExecutor	run raw JS in the page
TakesScreenshot	capture the page as an image
Cookie	one browser cookie
Window (manage().window())	size, position, maximize
Navigation (navigate())	back/forward/refresh
OutputType	FILE / BYTES / BASE64 for screenshots
Capabilities	describe a browser session (used with RemoteWebDriver)

MEMORY HOOK: *"WebDriver is the contract, the drivers are the implementations — and WebDriver itself being an INTERFACE is your polymorphism answer."*

SECTION 10 — THE TESTNG TOOLKIT (annotations, attributes, asserts)

Execution order (the hotel):

```
@BeforeSuite > @BeforeTest > @BeforeClass > @BeforeMethod > @Test > @AfterMethod
> @AfterClass > @AfterTest > @AfterSuite
```

@Test attributes (the control knobs):

Attribute	Does
`priority = 1`	run order (lowest first, default 0)
`dependsOnMethods = "x"`	run only after x passes (skipped if x fails)
`alwaysRun = true`	run even if the dependency failed
`enabled = false`	switch a test off
`invocationCount = 5`	run it 5 times
`timeOut = 5000`	fail if slower than 5s (ms)
`expectedExceptions = X.class`	pass only if X is thrown
`dataProvider = "name"`	rows of test data
`description = "..."`	report text

Asserts:

Method	Use
`Assert.assertEquals(actual, expected)`	hard — halts the test
`Assert.assertTrue(cond)`	hard boolean check
`Assert.assertFalse(cond)`	hard negative
`Assert.assertNull(x)`	hard null check
`SoftAssert`	collects ALL failures, speaks at `assertAll()`
`soft.assertAll()`	without it, soft failures are SILENT

MEMORY HOOK: *"Hard = halt, soft = save all and spill at assertAll — Suite > Test > Class > Method for setup, exact reverse for teardown."*

SECTION 11 — LISTENERS & SCREENSHOT ON FAILURE (the code they ask for)

What a listener is: TestNG's CCTV camera — it watches the run and fires a method at every event (test starts, passes, fails, skips). It NEVER touches your test logic; it only reacts.

The listener family (know each by name):

Interface	Fires when	The famous methods
ITestListener	any test event	`onTestStart`, `onTestSuccess`, `onTestFailure`, `onTestSkipped`
IInvokedMethodListener	before/after EVERY invoked method	`beforeInvocation`, `afterInvocation`
ISuiteListener	suite begins / ends	`onStart`, `onFinish`
IExecutionListener	whole execution starts/ends	`onExecutionStart`, `onExecutionFinish`
IAnnotationTransformer	annotations load — edit them at runtime	`transform(...)` (e.g. force `parallel`)
IMethodInterceptor	before methods run — reorder/filter them	`intercept(List<IMethodInstance>)`
IReporter	after the whole run — build a custom report	`generateReport(...)`

The screenshot-on-failure code (write this in your sleep):

```
public class TestListener implements ITestListener {

    // the listener does NOT own the driver — pull it from the test class
    private WebDriver getDriver(ITestResult result) {
        return ((BaseTest) result.getInstance()).driver;
    }

    @Override
    public void onTestFailure(ITestResult result) {
        String name = result.getName() + "_" + System.currentTimeMillis();
        File src = ((TakesScreenshot) getDriver(result))
                        .getScreenshotAs(OutputType.FILE);
        try {
            FileUtils.copyFile(src, new File("./screenshots/" + name + ".png"));
        } catch (IOException e) {
            e.printStackTrace();
        }
        System.out.println("FAILED -> " + result.getName() + " : screenshot saved");
    }

    @Override
    public void onTestStart(ITestResult result) {
        System.out.println("START -> " + result.getName());
    }

    @Override
    public void onTestSuccess(ITestResult result) {
        System.out.println("PASS  -> " + result.getName());
    }

    // onTestSkipped / onStart / onFinish can stay empty — you only
    // @Override the events you care about
}
```

Plug it in — 2 ways:

```
// way 1: on the class
@Listeners(TestListener.class)
public class LoginTest extends BaseTest { ... }
```

```
<!-- way 2: in testng.xml — applies to the WHOLE suite -->
<listeners>
```

```
    <listener class-name="com.qa.listeners.TestListener"/>
</listeners>
```

Why the cast dance: the listener runs OUTSIDE the test class, so it has no `driver` variable — you reach into the test object (`result.getInstance()`) and grab its driver, THEN cast to `TakesScreenshot`.

Where ExtentReports hooks in: the same listener — `onTestStart` creates the test node, `onTestFailure` logs the error + attaches the same screenshot. One listener, two jobs.

MEMORY HOOK: *"Listener = CCTV — watches every test, never touches the logic, films the crime (onTestFailure) with a screenshot. Driver comes from the victim (result.getInstance()), then cast TakesScreenshot."*

SECTION 12 — COMMON EXCEPTIONS (the quick table)

Exception	Meaning in one line
NoSuchElementException	locator found nothing — never born
StaleElementReferenceException	found it, but the DOM re-rendered — born then killed
ElementClickInterceptedException	something is covering the element
ElementNotInteractableException	exists but hidden/off-screen
TimeoutException	the wait's budget expired
NoAlertPresentException	switched to an alert that isn't there
NoSuchFrameException / NoSuchWindowException	that frame/window id doesn't exist
SessionNotFoundException	driver used after quit()
InvalidSelectorException	XPath/CSS syntax error
WebDriverException	the parent of them all — read the message

MEMORY HOOK: *"NoSuch = never born; Stale = born but the page killed it."*

SECTION 13 — COMMANDS (Maven, Git, Grid — the daily ones)

```
# ===== MAVEN =====
mvn clean test              # the daily run (surefire picks up testng.xml)
mvn clean install            # build + put the jar in local repo
mvn dependency:tree          # debug jar conflicts
mvn test -DsuiteXmlFile=testng.xml

# ===== GRID (v4) =====
java -jar selenium-server.jar standalone          # all-in-one
java -jar selenium-server.jar hub                  # classic pair:
java -jar selenium-server.jar node                   # registers with hub on 4444

# ===== GIT =====
git pull && git add . && git commit -m "tests" && git push
```

SECTION 14 — POWER COMBOS (the daily recipes — memorize these five)

```
// 1. Wait then click (the most-written pair in Selenium)
new WebDriverWait(driver, Duration.ofSeconds(10))
    .until(ExpectedConditions.elementToBeClickable(By.id("submit"))).click();

// 2. Switch to the new window, work, come home
String parent = driver.getWindowHandle();
for (String h : driver.getWindowHandles())
    if (!h.equals(parent)) driver.switchTo().window(h);
// ... work ...
driver.switchTo().window(parent);

// 3. Count and verify all links on a page
List<WebElement> links = driver.findElements(By.tagName("a"));
System.out.println("Total links: " + links.size());

// 4. Custom dropdown: open, loop, click the match
driver.findElement(By.id("dd")).click();
for (WebElement o : driver.findElements(By.cssSelector("ul.dropdown li")))
    if (o.getText().equals("India")) { o.click(); break; }

// 5. Screenshot on failure (inside ITestListener.onTestFailure)
File src = ((TakesScreenshot) driver).getScreenshotAs(OutputType.FILE);
FileUtils.copyFile(src, new File("screens/" + result.getName() + ".png"));
```

FINAL MEMORY HOOK (the whole sheet in one line): *"FIND with By (id first), DRIVE with get/navigate, ACT with click/sendKeys, READ with getText/getAttribute, WAIT with explicit snipers, SWITCH for frames-alerts-windows, GESTURE with Actions, VERIFY with Assert, CLEAN with quit."*